

AI SOFTWARE HOUSE — NO-CODE MULTI-AGENT WORKFLOW IN n8n

Name: _____ Batch: _____ Date: _____

LESSON SNAPSHOT

Goal:	Build a 3-agent AI team (PM, Developer, QA) using n8n — no code required	Tool:	n8n (self-hosted, visual workflow builder)
Parts:	10 parts covering concepts, building, testing, and reflection	Pre-req:	n8n installed locally, Groq API key

Warm-Up: What Do You Already Know?

1. In n8n, what is a "node"?

- A. A line of Python code
- B. A visual block that performs one specific action (fetch data, send email, call AI, etc.)
- C. A type of database
- D. A JavaScript framework

Hint: Think of each node as one LEGO block that does one job.

2. What does "multi-agent" mean in AI?

- A. One AI doing everything
- B. Multiple specialized AI systems collaborating, each with a specific role
- C. A virus that spreads across computers
- D. A group chat between humans

Hint: Think of a software company — would one person do PM, coding, AND testing?

3. What is a "System Prompt" when configuring an AI?

- A. The user's question
- B. Instructions that define the AI's role, personality, and rules before it sees user input
- C. The AI's final answer
- D. An error message from the system

Hint: It is like a job description given to a new employee on day one.

Part 1: The AI Software House — Why Multi-Agent?

THE SCENARIO: "The Smart Software House"

Imagine a software company where humans only provide the **ideas**, and a team of three AI employees does all the work:

1. The **Product Manager (PM)** turns your vague idea into clear, numbered technical requirements.
2. The **Developer** reads the requirements and writes Python code.
3. The **QA Engineer** reviews the code against the requirements. If it passes — ship it! If it fails — send it back to the Developer with feedback.

Today, we will build this entire company inside n8n — without writing a single line of code.

Why Not Just Use One AI?

You could ask ChatGPT: "Write me a Snake game and make sure it has no bugs." But that is like asking one intern to be the PM, developer, AND tester. The result is often messy. When we **split responsibilities**, each AI focuses on one job and does it better.

THE ANALOGY: A Restaurant Kitchen

One Chef Model: One person takes the order, cooks the food, plates it, AND checks quality. Overwhelmed. Mistakes happen.

Multi-Agent Model: A **waiter** (PM) takes the order clearly. A **chef** (Developer) cooks it. A **food inspector** (QA) tastes it before serving. If it is bad, it goes back to the chef. Much more reliable!

KEY INSIGHT: A multi-agent system splits work across specialized roles. Each AI has ONE job and a strict persona. This produces better results than asking one AI to do everything.

ACTIVITY: Think & Write:

Give one example from real life where splitting work into specialized roles produces better results than one person doing everything:

Part 2: Meet the Team — Three AI Employees

Each of our three AI employees is just the **same LLM** (e.g., Llama 3 via Groq) given a **different System Prompt**. The System Prompt is like a **job description** — it tells the AI who it is and what rules to follow.

Role	System Prompt (Job Description)	What They Produce
PM	"You are a senior Product Manager. Convert the user's request into numbered technical requirements."	A numbered list of requirements
Developer	"You are a senior Python developer. Read the requirements and QA feedback. Return ONLY production-ready code."	Complete Python code
QA Engineer	"You are a strict QA engineer. If code meets ALL requirements, respond EXACTLY: APPROVED. Otherwise, explain what needs fixing."	"APPROVED" or a list of issues

OBSERVATION: Notice that the QA prompt uses the word **"EXACTLY"**. This is critical. We need a predictable keyword ("APPROVED") so n8n can automatically route the workflow — either stop or loop back. Vague responses like "looks good" would be harder to detect.

ACTIVITY: Fill in the Blanks:

All three AI employees use the same _____ model, but each gets a different _____ .

The PM reads the user's _____ and produces _____ .
The QA responds with exactly the word _____ if the code passes.

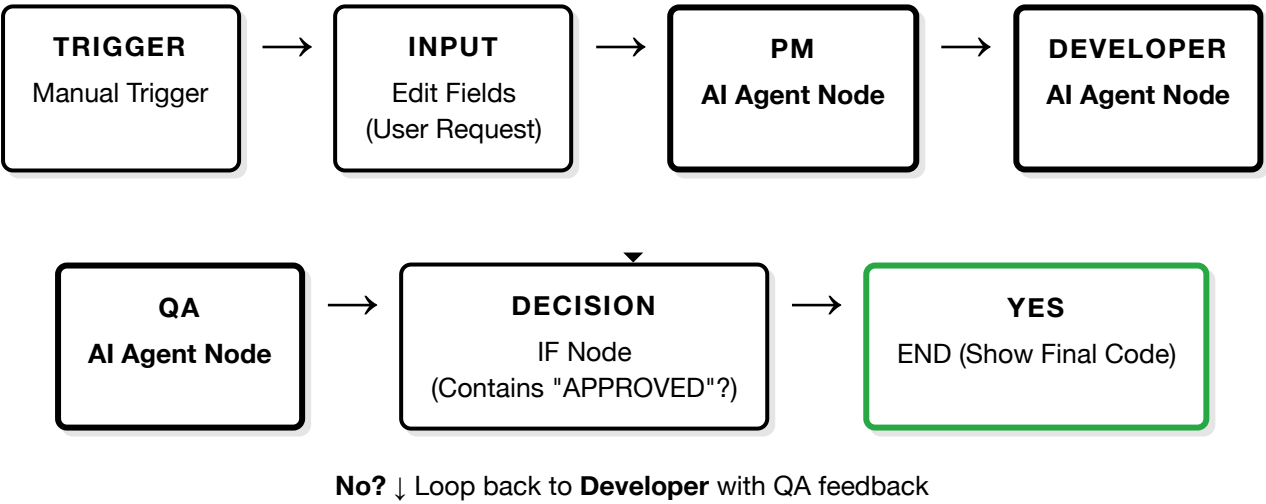
Part 3: How n8n Represents This — Nodes & Connections

In n8n, every step in a workflow is a **node**. You drag nodes onto the canvas and connect them with wires. Data flows from left to right through the connections.

The Node Types We Will Use

n8n Node	Our Use	What It Does
Manual Trigger	Start button	Kicks off the workflow when you click "Test Workflow"
Edit Fields (Set)	User's idea	Holds the user request text (e.g., "Build a Snake game")
AI Agent	PM / Developer / QA	Sends a prompt to an LLM and returns the AI's response
IF	QA Decision	Checks if the QA response contains "APPROVED" — routes to END or back to Developer

The Complete Workflow Flow



KEY INSIGHT: This is the **exact same architecture** as the Python/LangGraph version of this project — but built entirely by dragging and connecting visual blocks. No `StateGraph` , no `TypedDict` , no `.compile()` . Just nodes and wires.

How many AI Agent nodes do we need in this workflow?

- A. 1 (one AI does everything)
- B. 2 (PM and Developer share a node)
- C. 3 (one for PM, one for Developer, one for QA)
- D. 5 (one per step)

Part 4: Building the Workflow — Trigger & User Input

Let us start building! Open n8n in your browser (<http://localhost:5678>) and create a **new workflow**.

Step 1: Add a Manual Trigger

Click the **+** button on the canvas. Search for `Manual Trigger` and add it. This node has no configuration — it simply starts the workflow when you click "Test Workflow".

Step 2: Add an Edit Fields (Set) Node

Click the **+** after the trigger. Search for `Edit Fields` (also called "Set"). This node lets us define our user request.

EDIT FIELDS — CONFIGURATION

Field Name: `user_request`

Type: `String`

Value: `Build a Snake game in Python using Pygame with score tracking and collision detection.`

OBSERVATION: In the Python version, this was `user_req = st.text_area(...)` in Streamlit. In n8n, we simply type the text into a field. Same data, no code.

ACTIVITY: Write your own user request that you will use when testing. Make it specific enough for the PM to generate good requirements:

Part 5: Building the PM Agent Node

Now we add our first AI employee. Click **+** after the Edit Fields node, search for **AI Agent**, and add it. Rename it to `PM`.

PM — AI AGENT NODE CONFIGURATION

Node Name:	PM
Agent Type:	Tools Agent
System Message:	"You are a senior Product Manager. Convert the user's request into numbered technical requirements. Be specific and actionable."
Prompt (Text field):	{{ \$json.user_request }}

Sub-nodes: Adding a Chat Model

Every AI Agent node needs a **Chat Model** sub-node to specify which LLM to use. Click the AI Agent node, then click the **Chat Model** connector at the bottom.

CHAT MODEL — GROQ CONFIGURATION

Chat Model Node:	Groq Chat Model
Credential:	Your Groq API key (set up once in n8n Credentials)
Model:	llama-3.3-70b-versatile

THE ANALOGY: Hiring an Employee

Adding an AI Agent node = **hiring a new employee**.
The System Message = their **job description**.
The Chat Model = the **brain** they use to think (Groq/Llama 3).
The Prompt = the **specific task** you hand them on day one.

KEY INSIGHT: The expression `{{ $json.user_request }}` is n8n's way of saying "take the output from the previous node." It is similar to `state["user_request"]` in the Python version — but you never need to define a `TypedDict` or manage state manually. n8n passes data between nodes automatically.

What is the purpose of the System Message in an AI Agent node?

- A. It is the user's question
- B. It defines the AI's role, personality, and rules before it processes the prompt
- C. It displays an error message
- D. It starts the n8n server

Part 6: Building the Developer Agent Node

Add another **AI Agent** node after the PM. Rename it to `Developer` . Connect its Chat Model sub-node to Groq as before.

DEVELOPER — AI AGENT NODE CONFIGURATION

Node Name:

Developer

System Message:

"You are a senior Python developer. Read the requirements and any QA feedback. Return ONLY production-ready Python code."

Prompt:

Requirements: {{ \$('PM').item.json.output }}
QA Feedback: {{ \$('QA').item.json.output ?? 'None — first attempt' }}

Chat Model:

Groq — llama-3.3-70b-versatile

How does the Developer get the PM's output?

In n8n, you reference data from **any previous node by name**. The expression `{{ $('PM').item.json.output }}` means: "Go to the node named PM, get its output."

OBSERVATION: Notice the `?? 'None — first attempt'` part. This is n8n's **nullish coalescing** operator. The first time the Developer runs, QA has not produced anything yet. This provides a safe default value — exactly like `state.get('review', 'None')` in the Python version!

ACTIVITY: Match the n8n expression to its Python equivalent:

n8n Expression	Python Equivalent
1. {{ \$json.user_request }}	A. state.get('review', 'None')
2. {{ \$('PM').item.json.output }}	B. state["user_request"]
3. {{ \$('QA').item.json.output ?? 'None' }}	C. state["requirements"]

Your answers: 1→ 2→ 3→

Part 7: Building the QA Agent Node

Add a third **AI Agent** node after the Developer. Rename it to `QA`.

QA — AI AGENT NODE CONFIGURATION

Node Name:

QA

System Message:

"You are a strict QA engineer. If the code meets ALL requirements, respond EXACTLY with the word: APPROVED. Otherwise, explain what needs fixing."

Prompt:

Requirements: {{ \$('PM').item.json.output }}
Code: {{ \$('Developer').item.json.output }}
Review this code carefully.

Chat Model:

Groq — llama-3.3-70b-versatile

KEY INSIGHT: The QA agent is the **gatekeeper**. Its response determines whether the workflow stops or loops back. That is why the System Message says "respond EXACTLY with the word: APPROVED" — we need a

consistent, detectable keyword for the next step.

Why is it important that the QA responds with exactly "APPROVED" (and not "Looks good!" or "Yes, it works")?

- A. Because n8n only understands English words in capital letters
- B. Because we need a consistent, predictable keyword that the IF node can detect reliably
- C. Because the Developer node requires it
- D. Because Groq only accepts uppercase text

Part 8: The Decision Point — IF Node & Loop

This is the most important part. After QA responds, we need n8n to **decide**: Is the code approved? If yes, stop. If no, send it back to the Developer.

Step 1: Add an IF Node

After the QA node, add an **IF** node. This node evaluates a condition and routes data to one of two outputs: **True** or **False**.

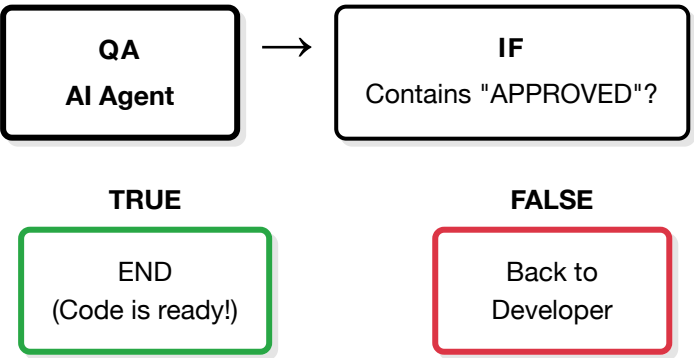
IF NODE — CONFIGURATION

Value 1:	{{ \$('QA').item.json.output }}
Operation:	Contains
Value 2:	APPROVED

Step 2: Connect the Outputs

The IF node has two output connectors:

- True (green):** Connect this to nothing — the workflow simply ends here. The final code is in the Developer's output.
- False (red):** Connect this **back to the Developer node** — this creates the loop!



THE ANALOGY: The Quality Checkpoint

This is like a factory assembly line with a quality checkpoint. Products that pass inspection go to the **shipping dock (END)**. Products that fail go back to the **repair station (Developer)**. The line keeps running until everything passes.

IMPORTANT: In n8n, when you connect the IF node's False output back to the Developer, you create a **loop**. Be careful — if the QA never approves, this loop could run forever! In practice, you could add a counter or a maximum iteration limit. For this worksheet, we trust our QA prompt to eventually approve.

ACTIVITY: Fill in the Blanks:

The IF node checks if the QA's output _____ the word "APPROVED".
If TRUE, the workflow _____.
If FALSE, data flows back to the _____ node.
This creates a _____ that repeats until the code passes QA.

Part 9: Running & Testing Your Workflow

Your workflow is now complete! Let us test it.

Step 1: Click "Test Workflow"

In the top-right corner of n8n, click the **Test Workflow** button. You will see each node light up green as it executes, one by one.

Step 2: Read the Execution Log

After the workflow finishes, click on any node to see its **output data**. This is how you inspect what each AI employee produced.

What to Check After Execution

Node	What to Look For	Signs of a Problem
PM	A numbered list of clear requirements	Vague or incomplete requirements
Developer	Complete Python code that addresses requirements	Incomplete code, missing imports, extra explanation text instead of pure code
QA	"APPROVED" or a specific list of issues	Vague feedback like "needs improvement" without details
IF	Data flowing to True (end) or False (loop)	Infinite loop — QA never approves

OBSERVATION: If the workflow loops back to the Developer, click on the Developer node's output for the **second iteration**. You will see improved code that addresses the QA's feedback. This is the power of the multi-agent loop — automatic self-correction!

ACTIVITY: Execution Trace: After running your workflow, fill in the actual results below:

PM produced requirements (how many?): _____
Developer iteration 1: Code had _____ lines
QA iteration 1 result: _____ (APPROVED or describe issue)

Did the workflow loop? _____ If yes, how many times? _____

Final code topic: _____

Part 10: Comparing Approaches — Code vs No-Code

We have now built the same "AI Software House" in two ways. Let us compare them.

Aspect	Python + LangGraph	n8n (No-Code)
State Management	Define a <code>TypedDict</code> with fields like <code>user_request</code> , <code>code</code> , <code>review</code> , <code>approved</code>	Data flows between nodes automatically — no manual state definition needed
Agent Definition	Write a Python function per agent (<code>pm_node</code> , <code>developer_node</code> , <code>qa_node</code>)	Drag an AI Agent node, type a System Prompt, connect a Chat Model
Routing / Loop	Write a <code>should_continue()</code> function + <code>add_conditional_edges()</code>	Add an IF node + drag a wire back to the Developer
Dependencies	<code>pip install langchain langgraph langchain_groq streamlit</code>	Install n8n once — everything else is built-in
UI	Build manually with Streamlit (<code>st.button</code> , <code>st.text_area</code> , etc.)	n8n canvas IS the UI — visual and interactive out of the box
Debugging	Print statements, stack traces, reading logs	Click any node to see its input/output data visually
Customization	Unlimited — full programming language	Limited to what nodes can do (but covers most use cases)
Deployment	<code>streamlit run app.py</code>	Always running on <code>localhost:5678</code> (or activate the workflow)

KEY INSIGHT: Neither approach is "better." **No-code is faster for building** and prototyping. **Code gives more control** for complex, custom logic. The best developers know BOTH — they prototype in n8n and build production systems in code.

In the Python version, what does `should_continue()` do that the IF node replaces in n8n?

- A. It sends emails
- B. It checks if the code was approved and decides whether to loop back to the Developer or end the workflow
- C. It installs Python packages
- D. It writes the code

ACTIVITY: Reflection: When would you use n8n, and when would you prefer writing Python code? Give one example for each:

Use n8n when: _____

Use Python when: _____

Final Challenge: Design a Marketing Agency Workflow

THE CHALLENGE: "The AI Marketing Agency"

You want to design a multi-agent workflow in n8n for a marketing firm generating Ad Campaigns:

1. **Researcher:** Researches competitor products based on a given topic.

2. **Copywriter:** Writes a Facebook Ad based on the research.

3. **Editor:** Checks if the Ad is under 500 characters and uses emojis. If not, sends it back to the Copywriter.

Task 1: Design the System Prompts

Write the System Message for each AI Agent node:

Agent	System Message
Researcher	
Copywriter	
Editor	

Task 2: Draw the Workflow

Sketch the n8n workflow using boxes and arrows. Include the node types (Manual Trigger, Edit Fields, AI Agent, IF). Mark where the loop happens.

Task 3: Configure the IF Node

What condition should the IF node check? Fill in the config:

IF NODE — YOUR CONFIGURATION

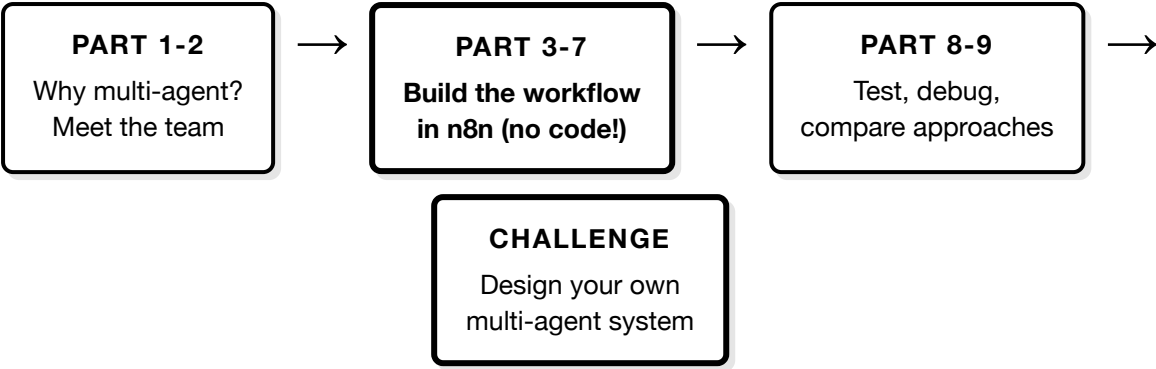
Value 1:

Operation:

Value 2:

Recap: Your Learning Map

What We Built Today



Key Concepts Checklist

Concept	I Can Explain It	n8n Equivalent
Multi-Agent Architecture	☐	Multiple AI Agent nodes, each with a unique System Prompt
System Prompt	☐	The "System Message" field in each AI Agent node
Data Passing (State)	☐	n8n expressions: <code>{{ \$('NodeName').item.json.output }}</code>
Conditional Routing	☐	The IF node with True/False outputs
Feedback Loop	☐	IF False output wired back to Developer node

ACTIVITY: Final Reflection: In your own words, explain why separating AI work into PM, Developer, and QA roles produces better results than asking one AI to "write code and make sure it works."